Stellar Wallet Extension — PRD & Design Briefs

Draft v0.1 · October 15, 2026 · Author: Enoch · Status: for review

1. Summary

Working name: [TBD]

A non-custodial browser-extension wallet for the Stellar network. Version 1 is deliberately small: create or import a wallet, send and receive, manage assets, swap at the best available price, and connect safely to Stellar apps. A light portfolio view is included; the full trading dashboard follows later.

The extension is the first product. A mobile app, passkey smart accounts, and naira on/off-ramps come after the extension proves it can be trusted and used.

One-line pitch: a Stellar wallet that explains itself, protects you from bad transactions, and shows what your money is doing.

2. Problem and opportunity

Research notes (as of October 2026, from public sources; verify before relying on them):

- Stellar already has several wallets. The most visible are Freighter (the Stellar Development Foundation's browser extension, which also has a mobile app), LOBSTR (the most widely used consumer wallet), xBull, Albedo, Rabet, and Hana. No single wallet dominates the way Phantom does on Solana.
- Freighter and LOBSTR already offer sending, receiving, trustline management, DEX swaps, Soroban signing, and scam-transaction scanning. These are baseline features, not differentiators.
- Swap routing is fragmented. Aggregation APIs exist (Soroswap's aggregation API, Stellar Broker), but no wallet pairs them with a good dashboard.
- Existing dashboards (explorers, analytics sites, a few open-source tools) are read-only and separate from the wallet.
- The Stellar DEX market is small compared with larger ecosystems, so trading volume alone will not sustain a product.

- A competing wallet (Neon Wallet) has an SCF-funded plan to add Stellar with passkey smart wallets, cross-chain swaps, and Blend access. Check its current status before finalizing positioning.

Opportunity: newcomers find Stellar's concepts confusing (reserves, trustlines, memos), and existing wallets mostly assume you already know them. A wallet that teaches while you use it, previews every transaction in plain language, and keeps your portfolio view next to your swap button can win a niche without outspending the incumbents.

3. Goals and non-goals

Goals (v1)

1. A user can create a wallet, back it up, fund it, and make a first payment in under five minutes without reading documentation.
2. Every transaction the user signs is shown in plain language before approval.
3. Swaps route through an aggregator for best price, with a clear summary of cost and slippage.
4. Any Stellar app can connect through a standard wallet interface.
5. Zero loss of user funds due to wallet bugs. Security outranks every other goal.

Non-goals (v1)

- Custody of user funds, or any server-side key handling.
- Fiat on/off-ramps, card payments, or any regulated money movement.
- Passkey smart accounts and fee sponsorship (planned for v2).
- NFT features, staking, lending, or cross-chain swaps.
- A mobile app.
- Support for other blockchains.

4. Target users

Persona A — The newcomer (primary). Holds some USDC or XLM, has used a bank app but not a crypto wallet. Is afraid of making a mistake that loses money. Needs plain language and guardrails.

Persona B — The Stellar-curious trader. Already holds a few Stellar assets, swaps occasionally, wants to see portfolio value and get a better price than the default.

Persona C — The builder. Develops or tests Stellar apps. Needs fast network switching (testnet and mainnet), clear signing details, and reliable dApp connections.

Persona A shapes the design. Persona C is a useful early tester and the source of the first bug reports.

5. Scope

Area	Feature	Priority	Release
Onboarding	Create wallet, recovery phrase backup and verification	P0	v1
Onboarding	Import wallet from recovery phrase	P0	v1
Security	Password lock, auto-lock, encrypted keystore	P0	v1
Accounts	Multiple accounts from one phrase	P1	v1
Balances	Asset list, XLM and token balances, available vs reserved	P0	v1
Send	Send XLM and assets with memo support	P0	v1
Receive	Address display, QR code, copy	P0	v1
Assets	Add and remove trustlines with reserve explanation	P0	v1
Swap	Aggregated best-price swap with preview	P0	v1
History	Transaction list with plain-language labels	P0	v1
Portfolio	Total value, 24-hour change, simple chart	P1	v1
dApp	Connection requests and transaction signing	P0	v1
Network	Switch between mainnet and testnet	P0	v1
Safety	Warning for destinations that require a memo	P1	v1
Safety	Contract-call simulation and plain-language preview	P0	v1
Safety	Third-party malicious transaction scanning	P1	Evaluate for v1.1

Area	Feature	Priority	Release
Hardware	Ledger support	P2	v1.1
Language	Hausa interface	P2	v1.1
Dashboard	PnL, price alerts, order history	P2	v1.2
Accounts	Passkey smart account, sponsored fees	P2	v2
Ramps	Naira on/off-ramp through anchors	P2	v2

6. Functional requirements

6.1 Onboarding and key management

- Generate a 12-word recovery phrase (BIP-39). Derive accounts using the standard Stellar path (SLIP-0010, m/44′/148′/n′). Verify against Stellar's reference tooling before release.
- The user must confirm the phrase by re-entering selected words before the wallet is usable.
- Keys are encrypted at rest with a user password and never leave the device or appear in logs.
- Import accepts a recovery phrase. Raw secret-key import is optional and warned about.
- Auto-lock after a configurable idle time. Lock on browser close.
- Show the recovery phrase again only after password re-entry.

6.2 Balances and accounts

- Show each asset with balance and, where a price exists, the fiat value.
- Explain Stellar's minimum balance (reserve) in plain language wherever it reduces what the user can spend, for example "1.5 XLM is reserved by the network for this account and 3 assets."
- Support multiple accounts, with editable names.
- Display addresses shortened with a copy action and a full-view option.

6.3 Send and receive

- Send flow: recipient, asset, amount, optional memo, review, confirm, result.
- Validate addresses before the review step. Support muxed addresses where the library allows it.
- Warn when the recipient is known to require a memo (SEP-29) and none is entered.
- Warn when the recipient account does not exist yet and explain the minimum funding amount.
- Review screen shows amount, fee, total, recipient, and memo in plain language.
- Receive screen shows the address, QR code, and a copy button, plus a note that only Stellar assets can be received.

6.4 Assets and trustlines

- Explain trustlines in one sentence: "Before you can hold an asset, your account must opt in to it."
- Search for assets by code or issuer. Show the issuer address and a verified indicator when the asset appears on a trusted list.
- Adding an asset shows the reserve cost. Removing an asset requires a zero balance and explains why.
- Warn about assets with unknown or recently created issuers.

6.5 Swaps

- Quotes come from an aggregation API. Show expected output, minimum received after slippage, route summary, fee, and price impact.
- Default slippage is conservative. Advanced settings allow changes with a warning above a threshold.
- Show a prominent warning for low-liquidity pairs and high price impact.
- Quotes expire after a short time. Show a countdown and require a refresh.
- If the user lacks a trustline for the output asset, offer to add it as part of the same flow.
- Fall back to direct Stellar DEX execution if the aggregator is unavailable, and tell the user.

6.6 dApp connection and signing

- Implement a standard wallet interface so apps can connect (target the Stellar wallet standard and verify the current SEP number, and ensure compatibility with the Stellar Wallets Kit).
- Connection request shows the site's origin, requested permissions, and the account to share.
- Signing requests show a plain-language summary first, with a collapsible technical view (operations and raw data).
- Soroban contract calls are simulated before the prompt appears, and the result is summarized: what will leave the account, what will arrive, and which contract is involved.
- A connected-sites list in Settings lets the user revoke access.
- Warn on signing requests that grant open-ended spending permission.

6.7 History

- List recent payments, swaps, trustline changes, and contract calls with readable labels ("Sent 20 USDC to GB4K…").
- Link each item to a block explorer.
- Cache history locally and refresh from the network.

6.8 Portfolio (light)

- Total value in the user's chosen display currency, 24-hour change, and a simple line chart per asset.
- Prices come from a market data provider through a thin proxy. If prices are unavailable, show balances without values and say so.

6.9 Settings

- Network (mainnet or testnet), display currency, auto-lock timer, connected sites, recovery phrase reveal, reset wallet, and an About page with version and open-source links.

7. Non-functional requirements

- **Security:** see section 9. No remote code execution, no analytics that capture addresses or balances without opt-in, strict content security policy.
- **Performance:** popup opens and shows balances from cache in under one second. Network data refreshes in the background.
- **Reliability:** work against more than one RPC endpoint with automatic fallback. Surface network errors in plain language.
- **Browser support:** Chrome and Chromium browsers (Manifest V3) for v1. Firefox in v1.1.
- **Accessibility:** keyboard navigation, visible focus, readable contrast in light and dark modes, no information conveyed by color alone.
- **Localization:** English first. Build all text through a translation layer from day one so Hausa can follow in v1.1.
- **Privacy:** no account creation, no email, no tracking of wallet activity by default.

8. Technical approach

- **Extension:** Manifest V3, React with TypeScript, a background service worker for signing and state, and a content script that exposes the wallet interface to web pages.
- **Stellar libraries:** the official Stellar JavaScript SDK for transaction building and RPC calls. Prefer well-maintained libraries over custom cryptography.
- **Keystore:** encrypted with a key derived from the user's password using a memory-hard function. Decrypted material lives only in the background worker's memory and is cleared on lock.
- **Data:** public RPC endpoints for balances and simulation. A thin backend (NestJS) is limited to price data and swap quotes so API keys stay off the client. The backend never sees keys, phrases, or signed secrets.
- **History:** the network RPC keeps limited history, so v1 queries a public explorer or indexer API and caches results locally.
- **Fork or build:** decide in discovery. Forking Freighter saves time but inherits its structure and license terms (check the repository license). Building fresh gives design freedom at higher security risk. Either way, reuse audited cryptographic libraries.
- **Distribution:** Chrome Web Store, with the source published so users and reviewers can verify what they install.

9. Security requirements and threat model

Threat	Mitigation
Phishing sites impersonating apps	Always show the requesting origin. Warn on newly seen or flagged domains.
Malicious transactions	Plain-language preview, simulation, spending-permission warnings, optional third-party scanning in v1.1.
Compromised dependency	Pin exact versions, review updates, minimal dependencies, reproducible builds.
Stolen device or malware	Encrypted keystore, auto-lock, no plaintext secrets in storage or logs.
Fake wallet copies in the store	Publish verifiable source and official links. Brand clearly.
Backend compromise	Backend holds no secrets that can move funds. Quotes are verified on-device before signing.
User error (wrong address, missing memo)	Address validation, memo warnings, review screen, test-send suggestion for large transfers.

Release gates for mainnet: independent code review, a bug-report channel, per-transaction and per-balance guidance for the beta, and a documented incident-response plan.

10. Success metrics

Targets are proposals to calibrate after the first beta.

- **Activation:** percent of installs that create or import a wallet and complete one transaction.
- **Time to first transaction:** target under five minutes for new users.
- **Retention:** percent of activated users who return after seven days.
- **Swap completion:** percent of started swaps that finish, and the median price improvement versus a default route.
- **Trust:** zero fund-loss incidents; time to fix reported security issues.

- **Comprehension:** in usability tests, most new users can explain what a trustline and a reserve are after using the wallet.

11. Monetization (decision pending)

Not part of v1 scope, but the design should not block it. Options to evaluate: a share of swap fees through the aggregator (confirm whether one is offered), referral fees from on/off-ramps once added, sponsored listings in a clearly labeled section, and premium dashboard features. A wallet with limited revenue is the likely outcome at first, so keep running costs near zero and sustain the project with grants and integration work.

12. Milestones (indicative, solo builder)

1. **Discovery and design (1–2 weeks):** decide fork or build, finish wireframes, set up testnet environment.
2. **Core wallet on testnet (3–5 weeks):** onboarding, keystore, balances, send, receive, trustlines, history.
3. **Swaps and dApp connection (3–4 weeks):** aggregator integration, signing popups, simulation previews.
4. **Portfolio and hardening (2–3 weeks):** price view, error states, accessibility pass, external review.
5. **Mainnet beta (ongoing):** limited release with clear caps, feedback loop, and fast fixes.

These estimates assume part-time solo work and will shift with the fork-or-build decision.

13. Risks and open questions

Risks

- A competitor ships similar features first. *Mitigation:* focus on clarity and the dashboard.
- A security bug harms users. *Mitigation:* small scope, review before mainnet, caps.
- Third-party APIs change or disappear. *Mitigation:* abstract providers and keep a fallback.
- Low revenue. *Mitigation:* near-zero costs, grants, services income.

Open questions

1. Fork Freighter or build fresh?
2. Which exact wallet interface standard and library version do target dApps expect?
3. Is there a fee-sharing arrangement available from the swap aggregator?
4. Which scam-scanning service is available and affordable for a small project?
5. What is the right name and brand?
6. Which RPC and history providers will be used in production?

14. Design briefs

14.1 Design principles

1. **Explain, don't assume.** Stellar terms (reserve, trustline, memo) get one-sentence explanations in context, never a glossary the user must find.
2. **Show what will happen.** Every approval screen says what leaves the account and what arrives.
3. **Calm by default.** Warnings are rare, so that when they appear, users take them seriously. Use red only for real danger.
4. **Small and fast.** The popup is a quick tool, not a dashboard. Heavy views open in a full tab.
5. **Accessible and local.** Keyboard-friendly, readable at small sizes, and ready for Hausa.

14.2 Visual direction

Trustworthy, quiet, modern. Plenty of whitespace, one accent color, a restrained neutral palette, and clear typographic hierarchy for amounts and addresses. Light and dark modes from the start. Avoid exchange-style visual noise and neon crypto clichés. Choose the accent color and wordmark during brand work, and test text contrast at small sizes.

Popup frame: about 360 × 600 px. **Full-tab view:** responsive, for onboarding, portfolio detail, and long reviews.

14.3 Screen map

Welcome → Create or Import → Backup and verify → Set password → Home

Home → Asset detail · Send · Receive · Swap · Add asset · History · Settings

External prompts: Connect request · Sign request · Unlock

14.4 Screen briefs

Welcome

- *Purpose:* make the first choice obvious and reassure.
- *Content:* wordmark, one-sentence promise, "Create a new wallet" (primary) and "I already have a wallet" (secondary).
- *Notes:* link to the open-source repository and a short "how your keys are stored" explainer.

Create wallet and backup

- *Purpose:* get the user to back up the phrase properly without frightening them.
- *Flow:* password → recovery phrase reveal (hold-to-reveal, no screenshots hint) → word-check quiz → success.
- *States:* wrong word, weak password, copy blocked, abandoned halfway.
- *Copy tone:* direct and kind. "This phrase is the only way to recover your wallet. We can't reset it for you."

Import wallet

- *Content:* phrase input with word-by-word validation, optional account count to scan.
- *States:* invalid word, wrong length, success with detected accounts.

Unlock

- *Content:* password field, unlock button, forgot-password path that explains a reset needs the recovery phrase.

Home (portfolio)

- *Purpose:* answer "how much do I have and what can I do?"
- *Content:* account name and network badge, total value with 24-hour change, a short asset list, and four primary actions (Send, Receive, Swap, Add asset).
- *States:* empty (new account with a gentle "fund your wallet" card), loading from cache, offline, prices unavailable.
- *Notes:* if the account is not yet funded on the network, say so plainly and show the address.

Asset detail

- *Content:* balance, value, price chart, issuer info and verification status, recent activity for that asset, and actions.

- *Notes:* show reserved vs spendable balance for XLM.

Send (four steps)

1. *Recipient:* address field with paste, validation, and recent contacts.
2. *Amount:* asset picker, amount with max button, spendable balance shown.
3. *Memo (optional):* shown only when relevant, with a warning if the recipient requires one.
4. *Review:* recipient (shortened plus full on tap), amount, fee, memo, network. A single confirm button.

- *Result:* success with a link to the explorer, or a clear failure with what to try next.
- *States:* invalid address, unfunded recipient, insufficient spendable balance, network error.

Receive

- *Content:* large QR code, full address with copy, account name, and the line "Only send assets on the Stellar network to this address."

Add asset (trustline)

- *Content:* search field, results with name, code, issuer, and verification badge, plus the reserve cost.
- *States:* no results, unknown issuer warning, insufficient XLM for reserve.

Swap

- *Content:* from and to pickers, amount, quote card (expected, minimum, rate, price impact, route summary, fee), slippage control tucked under advanced.
- *States:* loading quote, expired quote with refresh, low liquidity warning, missing trustline prompt, aggregator unavailable fallback notice.
- *Notes:* the confirm button names the action ("Swap 50 USDC for about 160 XLM").

History

- *Content:* grouped by day, readable labels, status chip, explorer link.
- *States:* empty, loading, failed fetch with retry.

Connect request (popup window)

- *Content:* site origin in large type, requested permissions in plain language, account selector, Approve and Reject.
- *States:* flagged domain warning, previously approved site.

Sign request (popup window)

- *Purpose:* the most important screen in the product.
- *Content:* a headline sentence ("This site will take 100 USDC from your account"), a what-leaves and what-arrives summary, network, fee, and an expandable technical view.
- *States:* simulation failed (explain and discourage approval), open-ended permission warning, unknown contract notice.
- *Notes:* the Approve button is never the only visually prominent element when a warning is shown.

Settings

- *Sections:* Security (auto-lock, reveal phrase, reset), Network, Currency, Connected sites, About.
- *Notes:* destructive actions need password re-entry and a plain explanation.

14.5 Components and states to design

Amount and address displays (with shortened, full, and copied states); asset row and asset avatar (with fallback); warning banner (info, caution, danger); transaction preview card; quote card; network badge; empty-state cards; skeleton loaders; toast messages; confirmation sheets.

Each interactive component needs default, hover, focus, pressed, disabled, loading, and error states.

14.6 Microcopy guidelines

- Short sentences, active voice, no jargon without an explanation next to it.
- Name actions by outcome: "Send 20 USDC" rather than "Confirm."
- Errors say what happened and what to do next. Never blame the user.
- Avoid hype and price predictions anywhere in the product.
- Write all strings as translatable keys. Keep sentences free of fragments joined in code, so Hausa translation works.

14.7 Accessibility checklist

Keyboard-only operation for every flow, visible focus rings, minimum text contrast of 4.5:1, touch targets large enough for trackpads and touchscreens, labels on all inputs, and warnings that do not rely on color alone.

14.8 Deliverables expected from design

1. Low-fidelity wireframes for all screens in 14.4.
2. A clickable prototype of onboarding, send, swap, and the signing prompt.
3. A small design system covering color, type, spacing, and the components in 14.5.
4. Light and dark versions of the final screens.
5. A usability test script for five first-time users covering onboarding and a first payment.

15. Appendix: assumptions to verify before build

- Current feature sets of Freighter, LOBSTR, and Neon Wallet's Stellar support.
- License terms of any code you plan to fork.
- The current wallet-interface standard and its version.
- Swap aggregator terms, rate limits, and any fee-sharing.
- Chrome Web Store policies for wallet extensions.
- Availability and pricing of transaction-scanning services.